

Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types Clustering (Agrupamento)

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Machine Learning: Clustering (Agrupamento)

O agrupamento (*clustering*) é um dos tipos de aprendizado não supervisionado (*unsupervised learning*). É semelhante à classificação: o objetivo é dar um rótulo a cada ponto de dados. No entanto, diferentemente da classificação, não recebemos nenhum exemplo de rótulo associado aos dados previamente.

Devemos inferir, a partir dos dados em si, quais pontos de dados pertencem ao mesmo cluster. Isso pode ser alcançado usando alguma noção de distância entre os pontos de dados; pontos de dados no mesmo cluster estão, de alguma forma, próximos uns dos outros.

1.1 K-Means Clustering

Um dos métodos de agrupamento mais simples é o agrupamento *k*-médias (*k-means*). Ele visa produzir um agrupamento que seja ideal no seguinte sentido:

- O centro de cada cluster é a média de todos os pontos no cluster.
- Qualquer ponto em um cluster está mais próximo do seu respectivo centro do que do centro de qualquer outro cluster.

O agrupamento *k-means* recebe primeiro o número desejado de clusters, digamos *k*, como um hiperparâmetro. Em seguida, para iniciar o algoritmo, *k* pontos do conjunto de dados são escolhidos aleatoriamente como os centros dos clusters. A partir disso, as seguintes fases são repetidas iterativamente:

1. Qualquer ponto de dado é definido para pertencer a um cluster, cujo centro está mais próximo dele.
2. Então, para cada cluster, um novo centro é escolhido sendo a média dos pontos de dados contidos ali.

Este procedimento é repetido até que os clusters não mudem mais. Esse tipo de algoritmo é chamado de algoritmo de Maximização de Expectativa (*Expectation-Maximization - EM*), o qual sabe-se que sempre converge.

1.2 Exemplo Simples

A biblioteca `scikit-learn` tem uma implementação deste algoritmo. Vamos aplicá-lo a um conjunto de "bolhas" (*blobs*) geradas aleatoriamente.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Gerando dados fictícios sem rótulos intencionais
X, y = make_blobs(centers=4, n_samples=200, random_state=0, cluster_std=0.7)

# Treinando o modelo
model = KMeans(4)
model.fit(X)

print(model.cluster_centers_)
```

1.2.1 Permutações de Rótulos e Acurácia

Ao obtermos um score (`accuracy_score`) para avaliar o *k-means*, podemos receber um número baixíssimo. Apesar dos agrupamentos estarem fisicamente corretos, os nomes (índices numéricos) dados arbitrariamente a cada cluster pelo algoritmo podem estar permutados em relação aos rótulos reais.

Para corrigir isso, renomeamos os clusters baseando-se no rótulo original mais comum dentro dele (`scipy.stats.mode`).

```
import scipy
def find_permutation(n_clusters, real_labels, labels):
    permutation = []
    for i in range(n_clusters):
        idx = labels == i
        new_label = scipy.stats.mode(real_labels[idx])[0]
        permutation.append(new_label)
    return permutation
```

2 Exemplos Complexos e Limitações do k-means

O algoritmo *k-means* pode ter dificuldades quando os clusters não possuem formatos convexos (como luas entrelaçadas ou anéis concêntricos), pois ele baseia as fronteiras do agrupamento em linhas estritamente lineares a partir do centro.

2.1 Agrupamento por Densidade (DBSCAN)

Para contornar o problema dos dados não-convexos, podemos usar algoritmos de agrupamento diferentes. O algoritmo DBSCAN é baseado em densidades e funciona bem em dados cuja densidade nos clusters é uniforme e bem delineada por espaços vazios.

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN

X, y = make_moons(200, noise=0.05, random_state=0)
model = DBSCAN(eps=0.3)
model.fit(X)
```

A boa notícia é que o DBSCAN não exige que o usuário especifique o número de clusters. Mas, agora o algoritmo depende de outro hiperparâmetro vital: um limite matemático para a distância chamado `eps`.

3 Hierarchical Clustering (Agrupamento Hierárquico)

O agrupamento hierárquico funciona primeiro colocando cada ponto de dado em seu próprio cluster solitário e, em seguida, fundindo os clusters passo a passo com base em alguma regra, até que restem apenas a quantidade estipulada de agrupamentos.

Com uma medida de distância original entre pontos $d(u, v)$, definimos a distância entre conjuntos através dos métodos (linkages):

- **Single:** usa a menor distância possível entre os membros.
- **Complete:** usa a maior distância possível.
- **Average:** a média ponderada de todas as distâncias.
- **Ward:** tenta minimizar a variância a cada passo de junção do cluster.

O resultado de ligações hierárquicas pode ser lindamente visualizado com *Heatmaps* pareados com *Dendrogramas* utilizando a biblioteca `seaborn.clustermap`.